

Infotact Technical Internship Program: Advanced Generative AI Project Specifications and Evaluation Criteria

The following document establishes the technical requirements, architectural directives, and sprint-based development roadmaps for three distinct software engineering projects. Designed for execution within a fast-paced technology environment in Bengaluru, Karnataka, these specifications serve as comprehensive blueprints for Generative AI engineering interns entering the Infotact startup ecosystem.

In 2026, the Generative AI landscape has moved beyond basic chatbots. Enterprise teams expect proficiency in Agentic Workflows (LangGraph/CrewAI), Ambient AI systems, and specialized Retrieval-Augmented Generation (RAG) that merges external unstructured data with internal tabular logic. The following projects focus on highly sought-after industry solutions across three critical niches: Healthcare (Ambient Clinical Scribes), Manufacturing (Prescriptive Maintenance Agents), and Supply Chain (Autonomous Disruption Monitors).

This document outlines the requisite tasks, technological paradigms, and evaluation criteria for successful internship completion. It is imperative to note that the assessment of these projects relies heavily on the transparency and consistency of the development lifecycle. **The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.**

Project 1: Healthcare - Ambient Clinical Scribe & Automated SOAP Note Generator

Executive Problem Statement

Physician burnout is at an all-time high, driven largely by the administrative burden of documenting patient encounters in Electronic Health Records (EHR). Doctors spend hours after their shifts typing notes instead of treating patients.

The objective of this project is to build an "Ambient AI Scribe." Utilizing Automatic Speech Recognition (ASR) and Generative AI, the system will securely transcribe a simulated doctor-patient conversation in real-time and automatically synthesize the transcript into a structured clinical SOAP (Subjective, Objective, Assessment, Plan) note, drastically reducing administrative overhead.

Business Objectives and Key Performance Indicators

The strategic vision is to seamlessly integrate AI into the clinical workflow without disrupting patient care. Success is measured by the Word Error Rate (WER) of the transcription and the clinical accuracy of the generated SOAP note. The system must demonstrate strict adherence to data privacy guidelines, extracting only medically relevant information while filtering out casual small talk.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Physician	Frictionless, hands-free documentation during consultations.	Records the patient visit securely via a web app. Reviews the AI-generated SOAP note, makes minor edits, and approves it for EHR integration.
Medical Coder	Accurate symptom extraction for billing.	Utilizes the structured "Assessment" section of the AI note to assign accurate ICD-10 medical codes.

Minimum Viable Product Specifications

The foundational feature is the Audio Ingestion & ASR Pipeline. The intern must utilize advanced speech-to-text models (like OpenAI Whisper) to transcribe multi-speaker medical audio.

The core deliverable is the LLM Synthesis Engine. The system must utilize a Large Language Model (e.g., Claude 3.5 Sonnet or GPT-4o) with strict, few-shot prompt engineering to accurately map conversational dialogue into the formal, highly structured JSON schema required for a medical SOAP note.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Backend & Audio Processing	Python, FastAPI	Required to handle streaming audio buffers and orchestrate asynchronous AI API calls.
Speech-to-Text (ASR)	Whisper API	State-of-the-art transcription that handles complex medical terminology and speaker diarization well.

LLM Orchestration	LangChain & Pydantic	Pydantic is crucial to force the LLM to return valid JSON matching the exact SOAP note data structures.
--------------------------	----------------------	---

Four-Week Engineering Roadmap

Week 1: Audio Ingestion and Speaker Diarization

The intern must initialize the FastAPI backend and integrate the ASR model. Using open-source mock medical audio datasets, the intern will build a pipeline that uploads audio, transcribes it, and implements basic speaker diarization (distinguishing between "Speaker 1: Doctor" and "Speaker 2: Patient").

Week 2: Prompt Engineering for Clinical Structuring

Week two focuses on data synthesis. The intern will design the system prompts to convert the raw transcript into a SOAP note. This involves instructing the LLM to identify symptoms (Subjective), vital signs mentioned (Objective), the doctor's diagnosis (Assessment), and prescribed medications/tests (Plan).

Week 3: RAG for ICD-10 Code Recommendation

The third week introduces an advanced feature: medical coding. The intern will build a lightweight RAG system containing a vector database of ICD-10 codes. The AI must cross-reference its generated "Assessment" against this database to automatically suggest the correct billing codes for the physician's review.

Week 4: Human-in-the-Loop Dashboard and Security

The final week is dedicated to the UI. The intern will build a React or Streamlit dashboard where the doctor can view the transcript alongside the generated SOAP note. A "Human-in-the-Loop" validation step is mandatory: the doctor must be able to edit the note before clicking "Finalize." The codebase must be committed to GitHub with all API keys strictly secured.

Project 2: Manufacturing & Industry 5.0 - Prescriptive Maintenance RAG Agent

Executive Problem Statement

While *Predictive* Maintenance alerts factory managers that a machine is vibrating abnormally and might fail, it leaves the engineers scrambling to read through thousands of pages of PDF machinery manuals to figure out *how* to fix it.

The objective of this project is to build an Agentic RAG system for *Prescriptive* Maintenance. When a simulated IoT anomaly is triggered, the AI agent will autonomously search through ingested technical manuals, cross-reference the specific error code, and output a step-by-step repair guide, complete with a list of required tools and spare parts.

Business Objectives and Key Performance Indicators

The strategic goal is to reduce Mean Time to Repair (MTTR). Success is quantified by the agent's retrieval accuracy—its ability to pinpoint the exact schematic or paragraph in a dense, 500-page PDF manual and accurately prescribe the solution without hallucinating safety procedures.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Maintenance Technician	Instant, context-aware repair instructions on the factory floor.	Receives an alert: "Motor B Overheating." Asks the AI chatbot for the standard operating procedure (SOP) to inspect the specific motor model.
Plant Manager	Orchestrating automation and minimizing downtime.	Uses the system to automatically draft maintenance work orders based on the AI's recommendations.

Minimum Viable Product Specifications

The application foundation is the Multimodal Document Parsing pipeline. Industrial manuals contain dense text, complex tables, and wiring diagrams. The intern must utilize specialized parsers (like LlamaParse) to retain the structural integrity of these documents before vectorization.

The core deliverable is the Intent-Based Orchestrator Agent. The system will use an LLM equipped with tools to query the vector database, read simulated real-time telemetry context, and synthesize a prescriptive repair plan.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Document Parser	LlamaParse	Optimized specifically to extract complex layouts and tables from PDFs, which standard OCR tools corrupt.
Agent Framework	LangGraph / Google ADK	Enables intent decomposition, allowing the agent to plan multiple steps (e.g., Step 1: Check error code, Step 2: Find repair steps).

Vector Database	Qdrant or Chroma	For storing and retrieving the embedded manual chunks efficiently.
------------------------	------------------	--

Four-Week Engineering Roadmap

Week 1: Ingesting Complex Machinery Manuals

The project initiates with building the RAG foundation. The intern will collect sample industrial manuals (e.g., HVAC or motor maintenance guides). They must build a pipeline that parses these PDFs, chunks them with overlapping windows to preserve context, generates embeddings, and upserts them to the vector database.

Week 2: Simulated IoT Telemetry Integration

Week two focuses on context gathering. The intern will create an endpoint that receives mock IoT alerts (e.g., `{"machine_id": "PUMP-01", "error_code": "E-404", "temp": 105}`). The system must parse this alert to formulate a search query for the RAG system automatically.

Week 3: Agentic Reasoning and Prescriptive Generation

The third week introduces the reasoning agent. Using LangGraph, the intern will build a workflow where the agent receives the IoT alert, queries the manual for the specific `error_code`, and extracts the exact repair steps. The prompt must strictly enforce that the agent cites the page number of the manual it sourced the instructions from.

Week 4: Tool Use for Parts Ordering & UI Deployment

The final sprint involves extending the agent's capabilities via Tool Calling. The intern will write a mock API function called `check_inventory(part_id)`. If the manual states a belt needs replacing, the agent must autonomously call this function to check if the belt is in the warehouse. The final interactive terminal or UI must be documented and pushed to GitHub.

Project 3: Logistics & Supply Chain - Autonomous Disruption Monitoring Agent

Executive Problem Statement

Global supply chains are incredibly fragile. A sudden port strike, a localized weather event, or a geopolitical conflict can cripple manufacturing if alternative suppliers are not immediately sourced. Supply chain managers cannot manually monitor global news 24/7 to predict how an event on the other side of the world will impact their Tier-2 or Tier-3 suppliers.

The objective of this project is to build an Autonomous Supply Chain Disruption Monitor using a multi-agent Generative AI framework. The system will continuously scrape global news feeds, use LLMs to classify the severity of supply chain risks, map the disruption to specific suppliers in a mock database, and generate an actionable mitigation plan.

Business Objectives and Key Performance Indicators

The strategic vision is to achieve extreme supply chain resilience. Success is measured by the system's reasoning capabilities—specifically its ability to correctly trace the "blast radius" of a disruption down a multi-tier supply chain network and synthesize alternative sourcing options quickly.

User Personas and Operational Workflows

Persona	Primary Operational Needs	System Interaction and Workflow
Chief Supply Chain Officer (CSCO)	High-level risk assessment and rapid crisis response.	Reviews the AI-generated daily risk summary. Approves the AI's suggested alternative sourcing plan during a crisis.
Procurement Analyst	Automated mapping of global events to local inventory.	Queries the agent: "How does the strike in Port X affect our Q3 microchip inventory?"

Minimum Viable Product Specifications

The foundational feature is the OSINT (Open Source Intelligence) Data Ingestion. The intern must utilize web scraping tools or news APIs to feed real-time articles into the system.

The core deliverable is the Multi-Agent Supply Chain Framework. The system requires at least three distinct AI personas: a Risk Manager Agent to score the severity of the news, a Network Visualizer Agent to determine which suppliers are impacted, and an Alternative Sourcing Agent to recommend backup vendors.

Architectural Directives and Technology Stack

Component	Technology	Architectural Rationale
Agent Orchestration	CrewAI or LangGraph	CrewAI provides an excellent structure for defining specialized agents with distinct roles, goals, and backgrounds.
Search / Data Tools	Tavily Search API	An AI-optimized search engine that allows the agents to scour the web for real-time news and supplier data.
Data Processing	Python, NetworkX	NetworkX can be utilized to mathematically model the supply chain tiers as a graph.

Four-Week Engineering Roadmap

Week 1: Web Search Integration and Disruption Classification

The intern will initialize the Python environment and integrate a web search tool (like Tavily or DuckDuckGo Search). The primary task is creating a script that fetches recent logistics news and uses an LLM to classify the articles into categories (e.g., "Natural Disaster," "Labor Strike," "Safe").

Week 2: Building the Mock Supply Chain Knowledge Graph

Week two focuses on business logic. The intern must create a mock database (JSON or SQLite) representing a multi-tier supply chain (e.g., Tier 1: Final Assembly, Tier 2: Microchip Supplier in Taiwan, Tier 3: Silicon mine).

Week 3: Defining the Agent Roles and Tasks

The third week transitions to multi-agent architecture. Using CrewAI or LangGraph, the intern will define the Risk Manager Agent, the Network Visualizer Agent, and the Alternative Sourcing Agent. The intern will write the logic connecting them sequentially: the Risk Agent identifies a threat, passes the location to the Network Agent (which searches the database for suppliers in that region), which then passes the affected parts to the Sourcing Agent.

Week 4: Structured Reporting and Automated Playbooks

The final week is dedicated to actionable outputs. The agents must be constrained using Pydantic to output their findings in a strict, standardized JSON format. The intern will build a script that converts this JSON into an executive-level Markdown or HTML "Disruption Action Plan." The codebase must be tracked meticulously on GitHub, showing the iterative prompt tuning required to make the agents collaborate smoothly.

Universal Generative AI Engineering and GitHub Version Control Guidelines

To simulate a rigorous, enterprise-grade AI research and development environment indicative of top-tier deep tech companies in Bengaluru, strict adherence to modern MLOps practices and version control is an absolute requirement. The core tenet of this internship program is continuous, transparent, and high-quality contribution. AI Engineering is an iterative, experimental process, and your version control history must reflect this.

Mandatory Evaluation Protocol

The evaluation of the intern's performance and the final project submission is governed by a strict, non-negotiable directive regarding version control consistency.

The evaluation will only be done if the intern is having all 4 weeks of github commits and contributions.

A project submitted with a compressed commit history—for example, uploading the entire codebase in a massive, monolithic push during the final week or on the final day—will result in immediate disqualification. Professional engineering is a continuous process of iteration, and the version control history must accurately reflect daily progress, prompt refinements, agent architecture pivots, and debugging efforts.

GitHub Operational Standards and Security Practices

To pass the technical evaluation, interns must internalize and strictly adhere to the following Git and GitHub operational standards to maintain a pristine, industry-standard commit history.

1. Setup GitHub Projects (Kanban Board)

At the start of Week 1, the intern must navigate to their repository's **Projects** tab and create a new Kanban board (To Do, In Progress, Done).

- **Mandatory Action:** The intern must break down the 4-week roadmap into individual GitHub Issues (e.g., Issue #1: "Integrate Whisper API for ASR", Issue #2: "Tune prompts for Risk Manager Agent").
- **Timestamping:** GitHub automatically timestamps when an issue is created, moved to "In Progress," and closed. Managers will audit this board to verify a steady, weekly cadence of AI development.

2. Commit Frequency, Granularity, and Semantic Messaging

A commit should act as a logical wrapper for related changes; fixing a UI bug and updating an LLM prompt should produce two separate commits.

- **Mandatory Action:** Interns are expected to commit code frequently (3-5 times per active development day). Every commit message must reference the specific GitHub Issue it solves.
- **Format Example:** feat: added LangGraph routing for Alternative Sourcing Agent (fixes #5) or prompt-tuning: refined Pydantic schema to fix SOAP note hallucinations (fixes #8).

3. Branching Strategy and Clean History Management

Direct commits to the `main` or `master` branch are entirely forbidden. Interns must adopt a structured branching model, ensuring that all new RAG pipelines, agent tools, or UI features are developed on isolated, short-lived feature branches. At the end of each week, the intern must open a Pull Request against the `main` branch summarizing their architectural decisions.

4. Absolute Data Privacy and Credential Security

Security hygiene is paramount in Generative AI. **Hardcoding OpenAI, Anthropic, Tavily, or Pinecone API keys within the Python source code is grounds for immediate failure.** All sensitive data must be injected via secure Environment Variables (`.env` files) and the `.env` file must be strictly excluded from the repository using `.gitignore`. Furthermore, interns must ensure that no sensitive healthcare (PHI) or proprietary supply chain data is accidentally pushed to public repositories.

By strictly enforcing these comprehensive specifications, the program ensures that deliverables transcend mere API wrappers, transforming into robust, scalable, and highly maintainable Agentic AI applications indicative of elite industry standards.